Milestone 1 : EDA Report

Introduction

This report examines 79,330 reservations from the City Hotel dataset (July 2015 to August 2017) across 31 variables. We first check the schema, missing values, duplicate records, and recording errors. We then profile the numerical distributions and study the two supervised targets: booking cancellation for classification and average daily rate (ADR) for regression. The final sections audit all columns for target leakage at booking time and evaluate eight candidate features for customer segmentation.

1 Environment Setup and Data Loading

Code cell 1: Library imports, plotting configuration and dataset loading.

```
import os
import urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['font.family'] = 'serif'
plt.rcParams['mathtext.fontset'] = 'cm'
plt.rcParams['figure.dpi'] = 120
plt.rcParams['axes.titlesize'] = 13
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 11
plt.rcParams['axes.labelweight'] = 'bold'
plt.rcParams['xtick.labelsize'] = 10
plt.rcParams['ytick.labelsize'] = 10

file_path = None
for candidate in ['../data/raw/H2.csv', 'data/raw/H2.csv', 'H2.csv', '/content/H2.csv']:
    if os.path.exists(candidate):
        file_path = candidate
        break

if file_path is None:
    url = 'https://raw.githubusercontent.com/afaq-ahmad/Hotel-booking-demand-Case-Study/master/H2.csv'
    urllib.request.urlretrieve(url, 'H2.csv')
    file_path = 'H2.csv'

df = pd.read_csv(file_path)
print(f'Dataset successfully loaded from: {file_path}')
print(f'Total records: {df.shape[0]:,}')
print(f'Total variables: {df.shape[1]}')

```

Dataset size and loading

The City Hotel file loads 79,330 records across 31 variables. Each row corresponds to a single reservation.

2 Dataset Overview and Schema

Code cell 2: Structure of every variable: completeness, data type, cardinality and a sample value.

```
schema_df = pd.DataFrame({
    'Column': df.columns,
    'Non-Null Count': df.notnull().sum().values,
    'Null Count': df.isnull().sum().values,
    'Null Pct (%)': np.round(df.isnull().mean().values * 100, 2),
    'Data Type': df.dtypes.astype(str).values,
    'Distinct Values': df.nunique().values,
    'Sample Value': [df[col].dropna().iloc[0] if len(df[col].dropna()) > 0
    ↪ else 'N/A' for col in df.columns]
})
display(schema_df)
print(f'Memory usage: {df.memory_usage(deep=True).sum() / (1024 * 1024):.2f}
    ↪ MB')

```

Output

	Column	Non-Null Count	Null Count	Null Pct (%)	Data Type	Distinct	Sample Value
9	Children	79326	4	0.01	float64	4	0.0
11	Meal	79330	0	0.00	object	4	' HB '
12	Country	79306	24	0.03	object	166	'PRT'
21	DepositType	79330	0	0.00	object	3	' No Deposit'
22	Agent	79330	0	0.00	object	224	'6'
23	Company	79330	0	0.00	object	208	'NULL'

Memory usage: 68.93 MB

Structure and variable types

Eighteen variables are numerical and thirteen are text (68.93 MB in memory). In the raw table, only children (4 records, 0.01%) and country (24 records, 0.03%) register standard nulls. Company and agent show no empty cells because their missing entries were exported as the literal string NULL, and several text columns carry leading or trailing whitespace. A few numeric columns have very small value sets and behave like discrete categories: cancellation and repeat-guest are binary (2 values), adults and babies take 5 values each, and children takes 4. Arrival dates are split across year, month, week, and day columns, while reservation status date is stored as text (864 distinct dates). No booking ID exists in the data.

Cardinality differs sharply across columns (Table 1). Agent (224 levels), company (208), and country (166) are too wide for one-hot encoding and require frequency encoding or rare-level grouping. Market segment (8), distribution channel (5), customer type (4), and deposit type (3) have compact level sets that fit dummy encoding, while ADR (5,405 values) is continuous.

Table 1: Distinct values for selected variables.

Variable	Stored as	Distinct	Variable	Stored as	Distinct
ADR	decimal	5,405	Country	text	166
Status date	text	864	Market segment	text	8
Lead time	integer	453	Sales channel	text	5
Agent ID	text	224	Customer type	text	4
Company ID	text	208	Deposit type	text	3

3 Data Quality, Missing Values and Anomaly Audit

Code cell 3: Whitespace cleaning, missing-value and placeholder audit, duplicate detection and logic checks.

```
string_cols = df.select_dtypes(include=['object']).columns
for col in string_cols:
    df[col] = df[col].str.strip()

explicit_nulls = df.isnull().sum()[df.isnull().sum() > 0]
sentinel_nulls = {}
undefined_vals = {}

for col in string_cols:
    null_count = (df[col] == 'NULL').sum()
    if null_count > 0:
        sentinel_nulls[col] = null_count
    undef_count = (df[col] == 'Undefined').sum()
    if undef_count > 0:
        undefined_vals[col] = undef_count

quality_data = []
for col in set(list(explicit_nulls.index) + list(sentinel_nulls.keys()) + list
    ↪ (undefined_vals.keys())):
    exp = int(explicit_nulls.get(col, 0))
    sen = int(sentinel_nulls.get(col, 0))
    und = int(undefined_vals.get(col, 0))
    tot = exp + sen + und
    pct = (tot / len(df)) * 100
    quality_data.append({
        'Variable': col,
        'Explicit NaNs': exp,
        'Sentinel NULLs': sen,
        'Undefined': und,
        'Total Missing': tot,
        'Missing Pct (%)': np.round(pct, 3)
    })

quality_df = pd.DataFrame(quality_data).sort_values(by='Total Missing',
    ↪ ascending=False).reset_index(drop=True)
display(quality_df)

num_duplicates = df.duplicated().sum()
pct_duplicates = (num_duplicates / len(df)) * 100
print(f'Identical row duplicates: {num_duplicates:,} ({pct_duplicates:.2f}%)')

zero_guests = ((df['Adults'] == 0) & (df['Children'].fillna(0) == 0) & (df['
    ↪ Babies'] == 0)).sum()
print(f'Zero-guest records (Adults + Children + Babies == 0): {zero_guests}
    ↪ ({(zero_guests / len(df)) * 100:.3f}%)')

unaccompanied = ((df['Adults'] == 0) & ((df['Children'].fillna(0) > 0) | (df['
    ↪ Babies'] > 0))).sum()
print(f'Unaccompanied minors (Adults == 0, Children/Babies > 0): {
    ↪ unaccompanied} ({(unaccompanied / len(df)) * 100:.3f}%)')

zero_nights = ((df['StaysInWeekendNights'] == 0) & (df['StaysInWeekNights'] ==
    ↪  0)).sum()
```

```
print(f'Zero-night stays (Weekend + Week nights == 0): {zero_nights} ({(
    ↪ zero_nights / len(df)) * 100:.2f}%)')

room_p = (df['ReservedRoomType'] == 'P').sum()
print(f'Phantom room type P records: {room_p}')

```

Missing values, duplicates and implausible records

Once padding spaces are stripped, six columns contain missing or placeholder entries (Table 2). Company ID is NULL in 75,641 bookings (95.350%), so the raw identifier adds little value and can be dropped or reduced to a binary flag. Agent ID is NULL in 8,131 records (10.250%), which marks direct bookings made without an agency. Country has 24 empty records (0.030%). The 4 bookings missing children (0.005%) are the exact same 4 records where distribution channel is Undefined (2 of which also list market segment as Undefined).

Table 2: Variables with missing or placeholder entries after whitespace cleaning.

Variable	Empty entries	Placeholder NULL	Undefined	Total missing	Share (%)
Company identifier	0	75,641	0	75,641	95.350
Agent identifier	0	8,131	0	8,131	10.250
Country	24	0	0	24	0.030
Children	4	0	0	4	0.005
Distribution channel	0	0	4	4	0.005
Market segment	0	0	2	2	0.003

Nearly a third of the dataset (25,902 rows, 32.65%) consists of exact duplicates across all 31 columns, which leaves 53,428 unique rows. Because there is no booking ID, we cannot tell log duplicates apart from multi-room group blocks entered as identical lines. Either way, rooms in a batch block share one outcome, so random splitting on the raw table would place identical copies in both train and test sets. Deduplicating before splitting prevents this leak.

Physical logic checks also catch three groups of invalid or unusual records: 167 bookings (0.211%) have zero total guests (including all 10 rows with phantom room code P), 223 bookings (0.281%) list children or babies with zero adults (390 total rows with zero adults), and 331 bookings (0.42%) list zero stay nights (all with zero ADR). Zero-guest and zero-night records should be removed before modeling.

4 Descriptive Statistics of Numerical Variables

Code cell 4: Summary statistics, skewness and kurtosis for all numerical variables.

```
num_cols = df.select_dtypes(include=[np.number]).columns
desc_stats = df[num_cols].describe().T
desc_stats['skewness'] = df[num_cols].skew()
desc_stats['kurtosis'] = df[num_cols].kurtosis()
desc_stats = desc_stats[['count', 'mean', 'std', 'min', '25%', '50%', '75%', '
    ↪ max', 'skewness', 'kurtosis']]
desc_stats = np.round(desc_stats, 2)
display(desc_stats)

```

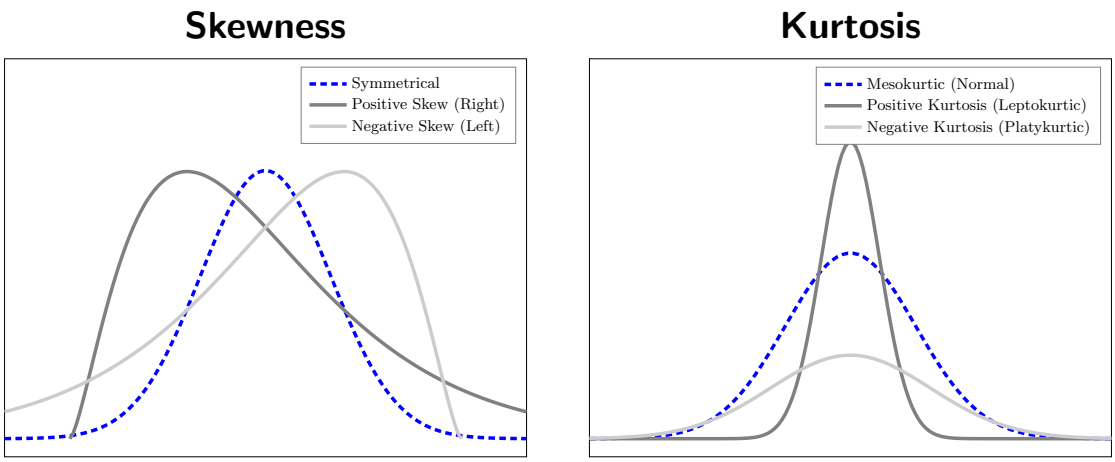


Figure 1: Reference illustration of skewness and kurtosis.

Central tendency, spread and tail behavior

Most of the 18 numerical variables are right-skewed with heavy upper tails (Figure 1, Table 3). Lead time averages 109.74 days against a median of 74.00 days (IQR 23.00 to 163.00, max 629, skewness 1.39). ADR averages 105.30 EUR (median 99.90, std 43.60), but a single 5,400.00 EUR entry drives its skewness to 23.16 and its kurtosis to 2,741.52. Four sparse history and modification counts (previous cancellations, previous non-canceled bookings, waiting list days, and booking changes) have a 75th percentile of zero alongside extreme maxima of 21, 72, 391, and 21. Two records listing 9 and 10 babies push babies kurtosis to 4,229.92. Because standard deviations span four orders of magnitude from 0.08 (babies) to 110.95 (lead time), linear and distance-based models require scaling and outlier handling.

Table 3: Selected summary statistics of numerical variables.

Variable	Mean	Median	Std. dev.	Maximum	Skewness	Kurtosis
Lead time (days)	109.74	74.00	110.95	629	1.39	1.80
ADR (EUR)	105.30	99.90	43.60	5,400	23.16	2,741.52
Weeknight stays	2.18	2.00	1.46	41	2.96	32.49
Weekend-night stays	0.80	1.00	0.89	16	1.13	5.00
Adults	1.85	2.00	0.51	4	−0.43	1.11
Children	0.09	0.00	0.37	3	4.34	18.78
Babies	0.00	0.00	0.08	10	44.78	4,229.92
Previous cancellations	0.08	0.00	0.42	21	16.58	451.56
Previous bookings not canceled	0.13	0.00	1.69	72	23.12	690.71
Booking changes	0.19	0.00	0.61	21	7.25	118.56
Days on waiting list	3.23	0.00	20.87	391	10.31	137.53
Special requests	0.55	0.00	0.78	5	1.40	1.66

5 Distributions of Key Variables

Code cell 5: Histograms with density curves for six main numerical variables (ADR restricted to values below 1,000).

```
key_num_vars = ['LeadTime', 'ADR', 'StaysInWeekNights', 'StaysInWeekendNights'
    ↪ , 'TotalOfSpecialRequests', 'BookingChanges']

fig, axes = plt.subplots(2, 3, figsize=(16, 9))
axes = axes.flatten()

for i, var in enumerate(key_num_vars):
    val_series = df[var].dropna()
    if var == 'ADR':
        val_series = val_series[val_series < 1000]
    sns.histplot(val_series, ax=axes[i], kde=True, color='steelblue', bins=30,
    ↪  edgecolor='white')
    axes[i].set_title(f'Distribution of {var}')
    axes[i].set_xlabel(var)
    axes[i].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

```

Shapes of the key distributions

The six histograms in Figure 2 divide into continuous right-skewed distributions and discrete counts. Lead time peaks near zero for short-notice bookings and tapers past 300 days, with small bumps between 150 and 350 days from advance tour blocks. Below 1,000 EUR, ADR sits mainly between 50 and 200 EUR around a 100 EUR mode, with sharp spikes at round prices (80, 100, and 120 EUR) and at 0 EUR (1,208 stays). Weeknight stays cluster at 1 to 3 nights and weekend stays sit at 0 to 2 nights, which matches typical short city trips. Special requests (60.45% zeros) and booking changes (84.88% zeros) are concentrated at zero, with very few bookings exceeding two requests or changes.

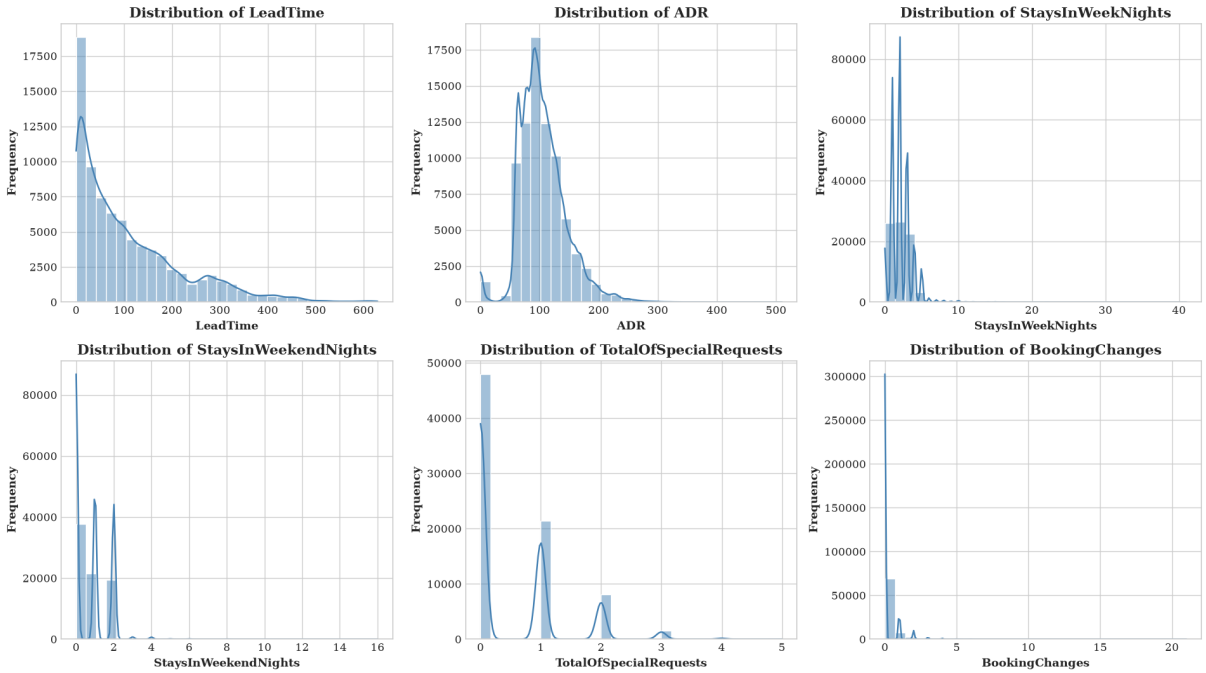


Figure 2: Histograms with density curves for lead time, ADR (values below 1,000), weeknight stays, weekend-night stays, special requests and booking changes.

6 Target Analysis: Cancellation Behavior

Code cell 6: Class balance of the cancellation outcome and cancellation rate by arrival month.

```
cancel_counts = df['IsCanceled'].value_counts()
cancel_pcts = df['IsCanceled'].value_counts(normalize=True) * 100

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

bars = ax1.bar(['Not Canceled (0)', 'Canceled (1)'], cancel_counts.values,
    ↪ color=['forestgreen', 'firebrick'], width=0.55, edgecolor='black',
    ↪ linewidth=0.8)
for bar in bars:
    height = bar.get_height()
    pct = (height / len(df)) * 100
    ax1.text(bar.get_x() + bar.get_width() / 2, height + 1000, f'{height:,}\n
    ↪ ({pct:.1f}%)', ha='center', va='bottom', fontsize=10, fontweight='bold'
    ↪ )
ax1.set_title('Overall Booking Cancellation Distribution')
ax1.set_ylabel('Number of Bookings')
ax1.set_ylim(0, 55000)

month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
    ↪  'August', 'September', 'October', 'November', 'December']
monthly_cancel = df.groupby('ArrivalDateMonth')['IsCanceled'].mean().reindex(
    ↪ month_order) * 100
ax2.plot(range(len(month_order)), monthly_cancel.values, marker='o', color='
    ↪ firebrick', linewidth=2.5, markersize=7)
ax2.set_title('Cancellation Rate by Arrival Month (%)')
ax2.set_xlabel('Arrival Month')
ax2.set_ylabel('Cancellation Rate (%)')
ax2.set_xticks(range(len(month_order)))
ax2.set_xticklabels(month_order, rotation=45, ha='right')
ax2.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

print(f'Not Canceled: {cancel_counts[0]:,} ({cancel_pcts[0]:.2f}%)')
print(f'Canceled: {cancel_counts[1]:,} ({cancel_pcts[1]:.2f}%)')

```

Class balance and monthly pattern

Of the 79,330 reservations, 46,228 (58.27%) were kept and 33,102 (41.73%) were canceled (Figure 3, left). Class imbalance is mild, so stratified splitting and standard classification metrics (ROC-AUC and F1) can be used without synthetic oversampling. Across the calendar year (Figure 3, right), monthly cancellation rates stay within a narrow 9.37-point band. Rates peak in spring and early summer during April (46.32%), June (44.69%), and May (44.38%), and dip lowest in March (36.95%), November (38.12%), and February (38.29

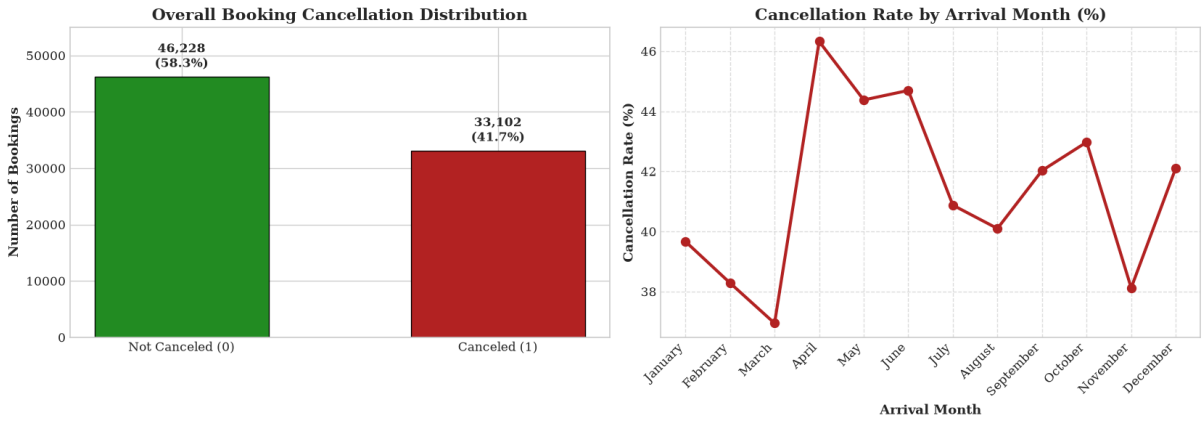


Figure 3: Left: number and share of bookings that were not canceled or canceled. Right: cancellation rate by arrival month.

7 Target Analysis: Average Daily Rate

Code cell 7: ADR quartiles, Tukey outlier fences, customer-type medians and bivariate pricing drivers.

```
q25, q50, q75 = df['ADR'].quantile([0.25, 0.5, 0.75])
iqr = q75 - q25
lower_bound = q25 - 1.5 * iqr
upper_bound = q75 + 1.5 * iqr
outliers = df[(df['ADR'] < lower_bound) | (df['ADR'] > upper_bound)]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
sns.boxplot(x=df[df['ADR'] < 1000]['ADR'], ax=ax1, color='lightblue')
ax1.set_title('ADR Boxplot (Filtered ADR < 1000 for Visual Clarity)')
ax1.set_xlabel('Average Daily Rate (EUR)')

customer_adr = df[df['ADR'] < 1000].groupby('CustomerType')['ADR'].median().
    ↪ sort_values()
sns.barplot(x=customer_adr.index, y=customer_adr.values, hue=customer_adr.
    ↪ index, ax=ax2, palette='Blues_r', legend=False)
ax2.set_title('Median ADR by Customer Type')
ax2.set_xlabel('Customer Type')
ax2.set_ylabel('Median ADR (EUR)')
plt.tight_layout()
plt.show()

df_adr_clean = df[(df['ADR'] > 0) & (df['ADR'] < 1000)].copy()
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
room_order = sorted(df_adr_clean['ReservedRoomType'].unique())
sns.boxplot(data=df_adr_clean, x='ReservedRoomType', y='ADR', order=room_order
    ↪ , hue='ReservedRoomType', ax=axes[0, 0], palette='Blues', legend=False)
axes[0, 0].set_title('ADR by Reserved Room Type (0 < ADR < 1000)')

seg_adr = df_adr_clean.groupby('MarketSegment')['ADR'].median().sort_values(
    ↪ ascending=False)
sns.barplot(x=seg_adr.index, y=seg_adr.values, hue=seg_adr.index, ax=axes[0,
    ↪ 1], palette='viridis', legend=False)
axes[0, 1].set_title('Median ADR by Market Segment')
axes[0, 1].tick_params(axis='x', rotation=30)

meal_adr = df_adr_clean.groupby('Meal')['ADR'].median().sort_values(ascending=
    ↪ False)
sns.barplot(x=meal_adr.index, y=meal_adr.values, hue=meal_adr.index, ax=axes
    ↪ [1, 0], palette='Blues_r', legend=False)
axes[1, 0].set_title('Median ADR by Meal Package')

monthly_adr = df_adr_clean.groupby('ArrivalDateMonth')['ADR'].median().reindex
    ↪ (month_order)
axes[1, 1].plot(range(len(month_order)), monthly_adr.values, marker='o', color
    ↪ ='navy', linewidth=2.2)
axes[1, 1].set_title('Median ADR by Arrival Month')
axes[1, 1].set_xticks(range(len(month_order)))
axes[1, 1].set_xticklabels(month_order, rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

Spread, outliers and pricing drivers

The middle half of ADR spans 79.20 to 126.00 EUR (IQR 46.80 EUR), which puts the Tukey outlier fences at 9.00 and 196.20 EUR (Table 4). Exactly 3,387 bookings (4.27%) fall outside those bounds. On the low end, 1,208 bookings (1.52%) have an ADR of 0.00 EUR from complimentary stays, zero-night records, or billing adjustments. On the high end, legitimate suite tariffs cluster between 196.20 and 510.00 EUR (Figure 4, left), whereas a single 5,400.00 EUR entry sits more than 121 standard deviations above the mean. Dropping the 5,400.00 EUR error and filtering zero-rate stays are required before fitting squared-error regression models.

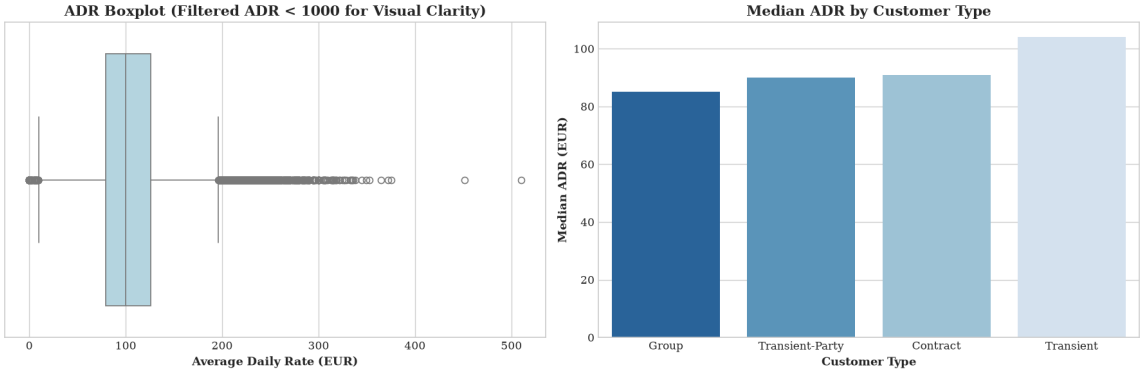


Figure 4: Left: boxplot of ADR for values below 1,000 EUR. Right: median ADR by customer type.

Table 4: Summary statistics and outlier bounds for ADR (EUR).

Mean	105.30	Interquartile range	79.20 to 126.00 (length 46.80)
Median	99.90	Tukey fences	9.00 to 196.20
Std. dev.	43.60	Outliers	3,387 (4.27%)
Min / Max	0.00 / 5,400.00	Zero-rate bookings	1,208 (1.52%)

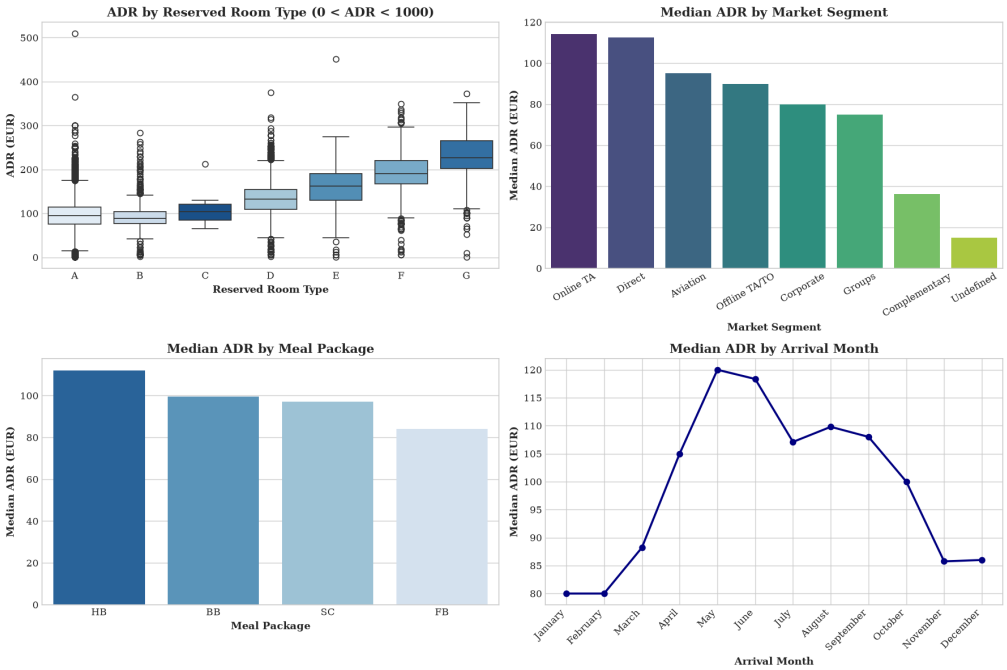


Figure 5: Bivariate pricing patterns for positive ADR below 1,000 EUR across reserved room type, market segment, meal package and arrival month.

Customer type creates a 19.00 EUR gap in median rate, from 85.00 EUR for groups and 90.00 EUR for transient-party bookings up to 90.95 EUR for contract and 104.00 EUR for individual transient guests (Figure 4, right). Room tier, sales channel, meal package, and season drive much larger price differences among positive rates below 1,000 EUR (Figure 5). Median ADR climbs steadily across room tiers from 88.77 EUR (B) and 94.50 EUR (A) to 132.30 EUR (D), 162.00 EUR (E), 190.00 EUR (F), and 227.25 EUR (G). Retail channels (Online TA at 114.30 EUR and Direct at 112.50 EUR) charge substantially more than negotiated volume segments (Offline TA/TO at 90.00 EUR, Corporate at 80.00 EUR, and Groups at 75.00 EUR). Half-board packages average a median of 112.10 EUR versus 99.45 EUR for bed-and-breakfast and 97.02 EUR for self-catering, while monthly medians rise by 50% from 80.00 EUR in January and February to 120.00 EUR in May and 118.35 EUR in June.

8 Rank Correlations Among Numerical Variables

Code cell 8: Spearman correlation matrix of numerical variables and ranking of correlations with cancellation.

```
corr_vars = [
    'IsCanceled', 'LeadTime', 'ArrivalDateYear', 'ArrivalDateWeekNumber',
    'ArrivalDateDayOfMonth', 'StaysInWeekendNights', 'StaysInWeekNights',
    'Adults', 'Children', 'Babies', 'IsRepeatedGuest',
    'PreviousCancellations', 'PreviousBookingsNotCanceled',
    'BookingChanges', 'DaysInWaitingList', 'ADR',
    'RequiredCarParkingSpaces', 'TotalOfSpecialRequests'
]

corr_matrix = df[corr_vars].corr(method='spearman')

plt.figure(figsize=(14, 11))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, cmap='coolwarm', vmin=-0.4, vmax=0.4,
    ↪ center=0, annot=True, fmt='.2f', linewidths=0.5, cbar_kws={'shrink':
    ↪ 0.8})
plt.title('Spearman Rank Correlation Matrix of Numerical Variables', fontsize
    ↪ =14, weight='bold', pad=15)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

top_corr_cancel = corr_matrix['IsCanceled'].drop('IsCanceled').sort_values(
    ↪ ascending=False)
display(pd.DataFrame({'Spearman Correlation with IsCanceled': np.round(
    ↪ top_corr_cancel, 3)}))

```

Associations with cancellation and among predictors

We use Spearman rank correlation (ρ) across the 18 numerical variables to measure monotonic associations without assuming linear relationships or normal tails (Figure 6). Six variables exceed $|\rho| = 0.10$ with cancellation (Table 5). Longer lead times (+0.325), prior cancellations (+0.280), and waiting-list days (+0.112) align with higher cancellation risk, whereas special requests (−0.324), booking changes (−0.196), and required parking spaces (−0.133) align with lower risk. Raw calendar variables (arrival year, week, and day) show almost zero rank correlation with cancellation ($|\rho| < 0.005$).

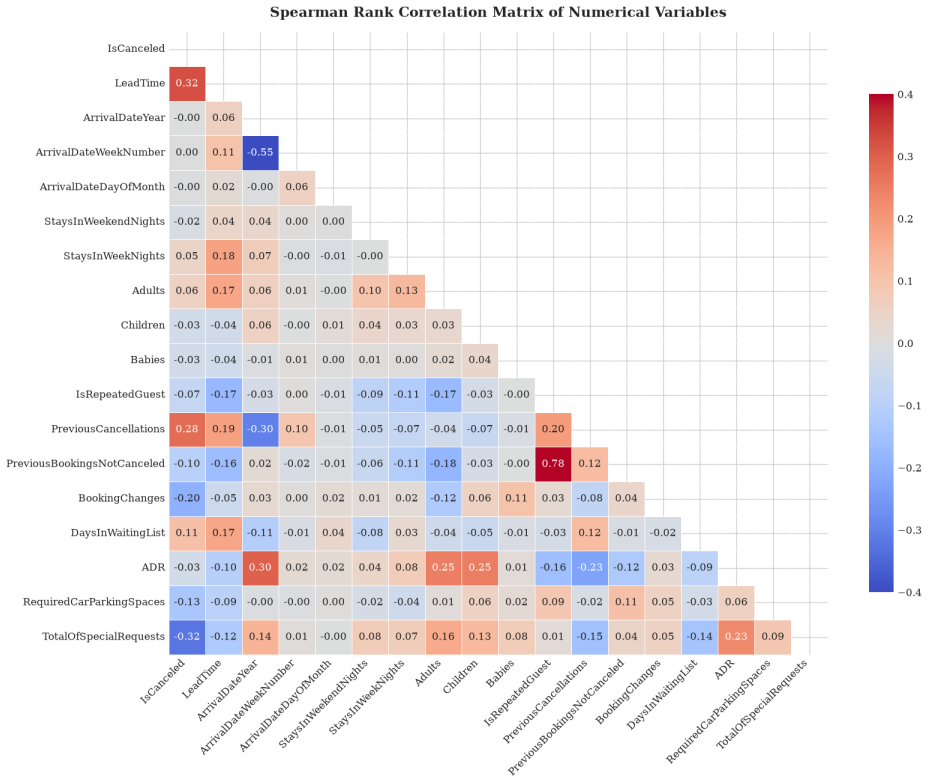


Figure 6: Spearman rank correlation matrix of the 18 numerical variables.

Table 5: Numerical variables with Spearman $|\rho| > 0.10$ with cancellation.

Variable	ρ	Variable	ρ
Lead time	+0.325	Special requests	−0.324
Previous cancellations	+0.280	Booking changes	−0.196
Days on waiting list	+0.112	Required parking spaces	−0.133

Among predictors, the repeat-guest flag and previous non-canceled bookings are largely redundant ($\rho = +0.78$). Arrival year and week number correlate negatively ($\rho = -0.55$) because the observation window starts in July 2015 and ends in August 2017, so 2015 covers only the second half of the year and 2017 covers only the first eight months. ADR increases with arrival year ($\rho = +0.30$), party size (adults +0.25, children +0.25), and special requests ($\rho = +0.23$).

9 Categorical Drivers of Cancellation

Code cell 9: Cancellation rate by commercial categories and top 10 guest origin countries.

```
cat_features = ['MarketSegment', 'DistributionChannel', 'DepositType', 'CustomerType']
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.flatten()

for i, col in enumerate(cat_features):
    group = df.groupby(col).agg(Total=('IsCanceled', 'count'), CancelRate=('IsCanceled', 'mean')).reset_index()
    group['CancelRate'] = group['CancelRate'] * 100
    group = group.sort_values(by='Total', ascending=False)
    sns.barplot(data=group, x=col, y='CancelRate', hue=col, ax=axes[i], palette='viridis', legend=False)
    axes[i].set_title(f'Cancellation Rate by {col}')
    axes[i].tick_params(axis='x', rotation=30)
    axes[i].set_ylim(0, 105)
    for p in axes[i].patches:
        axes[i].annotate(f'{p.get_height():.1f}%', (p.get_x() + p.get_width() / 2., p.get_height() + 2), ha='center', va='bottom', fontsize=9, fontweight='bold')
plt.tight_layout()
plt.show()

top10_countries = df['Country'].value_counts(dropna=False).head(10).index
country_stats = df[df['Country'].isin(top10_countries)].groupby('Country').agg(
    Bookings=('IsCanceled', 'count'),
    Share_Pct=('IsCanceled', lambda x: np.round(len(x) / len(df) * 100, 2)),
    Cancel_Rate_Pct=('IsCanceled', lambda x: np.round(x.mean() * 100, 2))
).loc[top10_countries]
display(country_stats)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
sns.barplot(x=country_stats.index, y=country_stats['Share_Pct'], hue=country_stats.index, ax=ax1, palette='Blues_r', legend=False)
ax1.set_title('Top 10 Origin Countries by Booking Share (%)')
sns.barplot(x=country_stats.index, y=country_stats['Cancel_Rate_Pct'], hue=country_stats.index, ax=ax2, palette='Reds_r', legend=False)
ax2.set_title('Cancellation Rate Among Top 10 Countries (%)')
plt.tight_layout()
plt.show()

```

Cancellation rates across booking categories and origin countries

Deposit policy produces the sharpest contrast in the dataset (Figure 7, Table 6): Non Refund bookings cancel at 99.81% (12,844 of 12,868) compared to 30.47% for No Deposit. As shown in Section 11, this pattern is an artifact of bulk group blocks, since 93.4% of Non Refund rows are exact duplicates canceled together. Third-party and group channels also cancel heavily: Groups reaches 68.86%, Offline TA/TO 42.83%, Online TA 37.40%, and the TA/TO channel 45.03%, whereas Direct bookings (17.33%) and Complementary stays (11.81%) stay low. By customer type, Contract (48.04%) and Transient (45.62%) cancel far more often than Transient-Party (28.10%) and Group (9.90%).

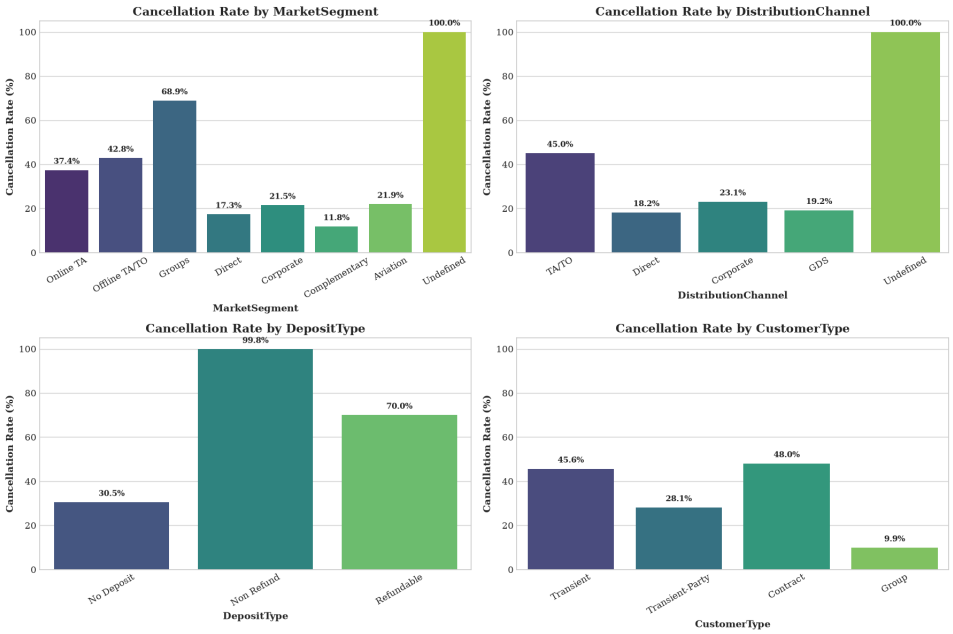


Figure 7: Cancellation rate by market segment, distribution channel, deposit type and customer type.

Table 6: Cancellations by deposit type.

Deposit type	Bookings	Canceled	Cancellation rate (%)
No Deposit	66,442	20,244	30.47
Non Refund	12,868	12,844	99.81
Refundable	20	14	70.00

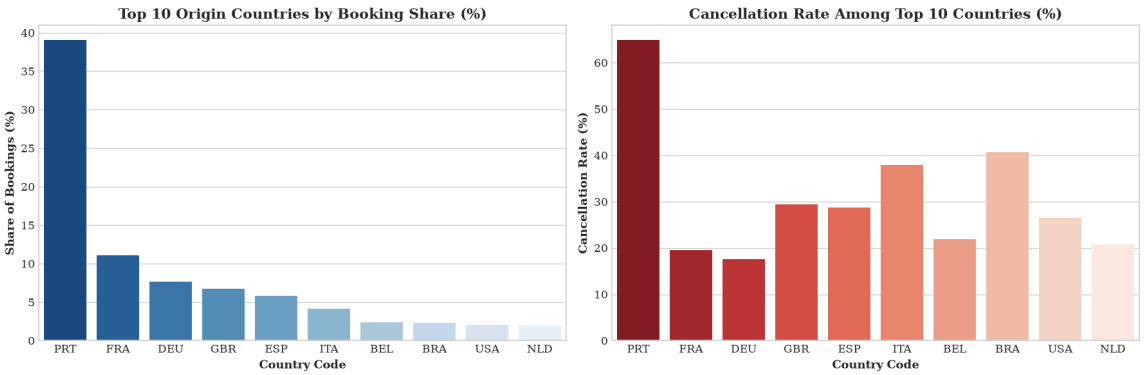


Figure 8: Left: share of total bookings for the top 10 guest origin countries. Right: cancellation rate within each top-10 country.

Table 7: Booking volume, dataset share and cancellation rate for the top 10 origin countries.

Country	Bookings	Share (%)	Cancel (%)	Country	Bookings	Share (%)	Cancel (%)
PRT	30,960	39.03	64.86	ITA	3,307	4.17	37.89
FRA	8,804	11.10	19.57	BEL	1,894	2.39	21.91
DEU	6,084	7.67	17.62	BRA	1,794	2.26	40.64
GBR	5,315	6.70	29.39	USA	1,618	2.04	26.51
ESP	4,611	5.81	28.76	NLD	1,590	2.00	20.82

Although guest origin spans 166 country codes, the top 10 countries account for 65,977 bookings, or 83.17% of the dataset (Figure 8, Table 7). Domestic reservations from Portugal (PRT) make up 39.03% of all records and cancel at 64.86%, whereas inbound travelers from Germany (17.62%), France (19.57%), the Netherlands (20.82%), Belgium (21.91%), the USA (26.51%), Spain (28.76%), and the UK (29.39%) cancel at less than half that rate. Keeping the top 10 to 15 countries and grouping the remaining codes into an Other category preserves this domestic-versus-foreign split without adding 166 sparse columns.

10 Lead Time, Special Requests and Room Assignment

Code cell 10: Room-assignment discrepancy indicator and cancellation patterns by lead time, special requests and room assignment.

```
df['RoomTypeDiscrepancy'] = (df['ReservedRoomType'] != df['AssignedRoomType']) .astype(int)

fig, axes = plt.subplots(1, 3, figsize=(17, 5))

sns.boxplot(data=df, x='IsCanceled', y='LeadTime', hue='IsCanceled', ax=axes [0], palette=['forestgreen', 'firebrick'], legend=False)
axes[0].set_title('Lead Time Distribution by Cancellation Status')
axes[0].set_xticks([0, 1])
axes[0].set_xticklabels(['Not Canceled (0)', 'Canceled (1)'])
axes[0].set_ylabel('Lead Time (Days)')

requests_cancel = df.groupby('TotalOfSpecialRequests')['IsCanceled'].mean() * 100
requests_cancel.plot(kind='bar', ax=axes[1], color='steelblue', edgecolor=' black')
axes[1].set_title('Cancellation Rate by Special Requests')
axes[1].set_xlabel('Total Special Requests')
axes[1].set_ylabel('Cancellation Rate (%)')
axes[1].tick_params(axis='x', rotation=0)

room_cancel = df.groupby('RoomTypeDiscrepancy')['IsCanceled'].mean() * 100
room_cancel.plot(kind='bar', ax=axes[2], color=['darkorange', 'forestgreen'], edgecolor='black')
axes[2].set_title('Cancellation Rate by Room Assignment Change')
axes[2].set_xlabel('Assigned != Reserved Room')
axes[2].set_xticks([0, 1])
axes[2].set_xticklabels(['Same Room (0)', 'Different Room (1)'], rotation=0)
axes[2].set_ylabel('Cancellation Rate (%)')

plt.tight_layout()
plt.show()

```

Behavioral and operational signals

Canceled bookings are reserved much further in advance than completed stays, with a mean lead time of 150.3 days (median 115.0, IQR 50.0 to 226.0, max 629) versus 80.7 days (median 50.0, IQR 12.0 to 121.0, max 518) for guests who check in (Figure 9, Table 8). Special requests show the opposite pattern: cancellation drops from 54.92% when guests make zero requests (47,957 rows) to 22.04% for one request and 3.85% for five requests. Room reassignments (where the assigned room differs from the reserved room, 9.07% of bookings) have a 5.81% cancellation rate compared to 45.31% when the room is unchanged. Because hotels reassign or upgrade rooms at check-in, the assigned room column leaks arrival information that is not known at booking time.

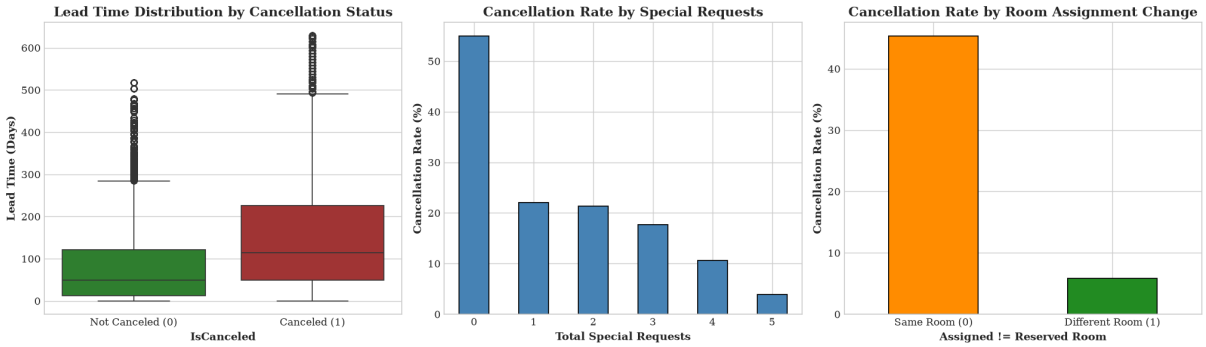


Figure 9: Left: lead time by cancellation outcome. Middle: cancellation rate by number of special requests. Right: cancellation rate when the assigned room differs from the reserved room.

Table 8: Lead time (days) distribution by cancellation outcome.

Outcome	Mean	Std. dev.	25th pct.	Median	75th pct.
Not canceled (0)	80.7	89.9	12.0	50.0	121.0
Canceled (1)	150.3	124.1	50.0	115.0	226.0

11 Target Leakage Audit

Code cell 11: Cross-tabulation of reservation status, required parking spaces and deduplicated deposit types against cancellation.

```
leakage_crosstab = pd.crosstab(df['ReservationStatus'], df['IsCanceled'], margins=True)
display(leakage_crosstab)

perfect_leakage = (
    ((df['ReservationStatus'] == 'Check-Out') & (df['IsCanceled'] == 0)).sum()  +
    ((df['ReservationStatus'].isin(['Canceled', 'No-Show'])) & (df['IsCanceled'] == 1)).sum()
)
print(f'Exact mathematical consistency: {perfect_leakage} / {len(df)} ({( perfect_leakage / len(df)) * 100:.1f}%)')

parking_crosstab = pd.crosstab(df['RequiredCarParkingSpaces'], df['IsCanceled'], margins=True)
display(parking_crosstab)

df_dedup = df.drop(columns=['RoomTypeDiscrepancy']).drop_duplicates()
print(f'Deduplicated rows: {len(df_dedup):,} (Canceled rate: {df_dedup["IsCanceled"].mean() * 100:.2f}%)')
display(pd.crosstab(df_dedup['DepositType'], df_dedup['IsCanceled'], margins=True))

```

Direct, check-in and batch-duplicate leakage across all tasks

We evaluate all predictors at the moment a booking is confirmed, prior to guest arrival. Reservation status is a direct restatement of the classification target across all 79,330 rows: Check-Out maps to 0, while Canceled (32,186 rows) and No-Show (916 rows) map to 1 (Table 9). Reservation status date logs when that final outcome was entered. Required parking spaces also leaks check-in activity: out of 1,926 bookings with 1 to 3 parking spaces, 0 were canceled because staff record parking spaces upon arrival (Table 10).

Table 9: Reservation status by cancellation outcome (number of bookings).

Reservation status	Not canceled (0)	Canceled (1)	Total
Canceled	0	32,186	32,186
Check-Out	46,228	0	46,228
No-Show	0	916	916
Total	46,228	33,102	79,330

Table 10: Required car parking spaces by cancellation outcome (number of bookings).

Required parking spaces	Not canceled (0)	Canceled (1)	Total
0 spaces	44,302	33,102	77,404
1 to 3 spaces (1: 1,921; 2: 3; 3: 2)	1,926	0	1,926
Total	46,228	33,102	79,330

Table 11: Deposit type by cancellation outcome after removing 25,902 exact duplicate rows ($N = 53,428$).

Deposit type (deduplicated)	Not canceled (0)	Canceled (1)	Total unique	Cancel rate (%)
No Deposit	37,350	15,218	52,568	28.95
Non Refund	24	821	845	97.16
Refundable	5	10	15	66.67
Total	37,379	16,049	53,428	30.04

Removing the 25,902 exact duplicate rows ($N = 53,428$) lowers the overall cancellation rate from 41.73% to 30.04% and cuts Non Refund volume from 12,868 rows to 845 unique rows (97.16% canceled), since 12,023 of those rows were duplicate group-block entries (Table 11). Classification models must drop reservation status, status date, assigned room type, and parking spaces. ADR regression and customer segmentation must drop all four columns along with the cancellation target.

12 Feature Associations for Customer Segmentation

Code cell 12: Pearson correlation matrix and statistics for eight candidate segmentation features.

```
cluster_features = [
    'LeadTime', 'ADR', 'StaysInWeekendNights', 'StaysInWeekNights',
    'Adults', 'Children', 'BookingChanges', 'TotalOfSpecialRequests'
]

cluster_corr = df[cluster_features].corr()

plt.figure(figsize=(9, 7))
sns.heatmap(cluster_corr, annot=True, fmt='.2f', cmap='Blues', cbar=True,
    ↪ linewidths=0.5)
plt.title('Correlation Matrix of Potential Customer Segmentation Features',
    ↪ fontsize=12, weight='bold', pad=12)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

cluster_summary = df[cluster_features].agg(['mean', 'std', 'min', 'median', '
    ↪ max']).T
display(np.round(cluster_summary, 2))

```

Redundancy and scale of candidate clustering features

Pairwise Pearson correlations across the eight candidate segmentation features stay at or below $|r| = 0.31$, with the strongest linear links connecting ADR to children (+0.31) and adults (+0.26) because larger parties book higher-priced rooms (Figure 10). This rules out linear collinearity among the eight features while still allowing nonlinear guest clusters to form in the joint space. However, feature scales differ by more than two orders of magnitude: standard deviations run from 0.37 (children) and 0.51 (adults) up to 43.60 (ADR) and 110.95 (lead time), with maxima at 5,400.00 EUR and 629.00 days (Table 12). Removing the 5,400.00 EUR outlier, capping extreme tails, and standardizing the features are necessary so lead time and ADR do not dominate Euclidean distance in PCA or K-Means.

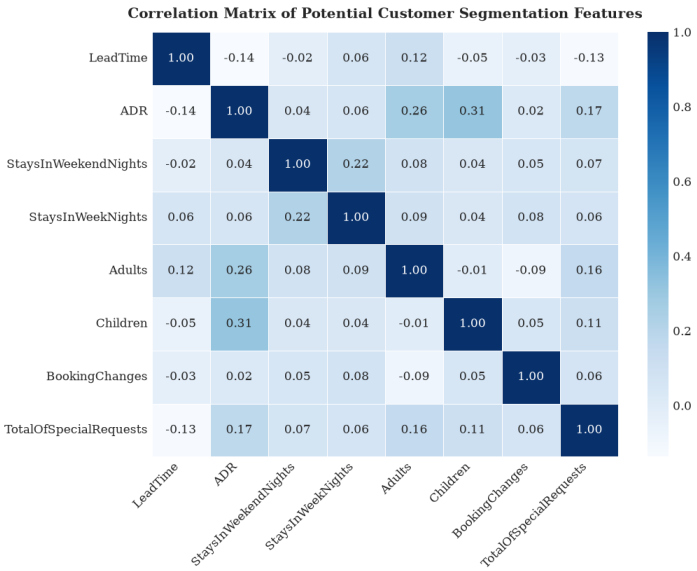


Figure 10: Pearson correlation matrix of eight candidate segmentation features.

Table 12: Summary statistics of the eight candidate segmentation features.

Feature	Mean	Std. dev.	Min	Median	Max
Lead time (days)	109.74	110.95	0.00	74.00	629.00
ADR (EUR)	105.30	43.60	0.00	99.90	5,400.00
Weekend-night stays	0.80	0.89	0.00	1.00	16.00
Weeknight stays	2.18	1.46	0.00	2.00	41.00
Adults	1.85	0.51	0.00	2.00	4.00
Children	0.09	0.37	0.00	0.00	3.00
Booking changes	0.19	0.61	0.00	0.00	21.00
Special requests	0.55	0.78	0.00	0.00	5.00